A Comparative Study of Polynomial Commitments and Verifiable Delegation Protocols

Angel Coronel
CSC 73010: Modern Cryptography
Computer Science, The Community University of New York

May 12, 2025

Abstract

This paper compares two cryptographic protocols: the polynomial commitment scheme by Kate, Zaverucha, and Goldberg (KZG), and the verifiable delegation of computation scheme by Benabbas, Gennaro, and Vahlis. These protocols consider efficient verification when an untrusted party does some computation for them. This paper will cover the assumptions made for the protocols, their cryptographic constructions and how they can be applied in the real world. Both papers are compared to view how they are similar and different in their complexity, efficiency and what kind of security guarantees they permit.

Contents

1 Introduction **3**

2 Background and Assumptions **3**
2.1 Notation and Bilinear-Group Preliminaries 3
2.2 Assumptions 3
2.3 Algebraic PRFs and "Closed Form Efficiency" (CFE) 4
2.4 Polynomial Commitment Assumptions (KZG) 4

3 Constructions Overview **5**
3.1 VD over Large Datasets 5
3.2 Constant-Size Polynomial Commitments 6
3.3 Side-by-Side Comparison 7

4 Applications **7**
4.1 Applications of KZG 8
4.2 Applications of Verifiable Delegation (VD) 9
4.3 Choosing the Right Tool 10

5 Discussion **10**

6 Conclusion **11**

7 References **12**

1 Introduction

Correctness and integrity is needed when outsourcing computation to untrusted servers. The KZG polynomial commitment scheme and the Benabbas et al. verifiable computation framework are two different mechanism that provide a solution for correctness and integrity. This paper analyses and compares both of these mechanism by viewing their design techniques and applicability.

2 Background and Assumptions

Here the security foundations are built for the two protocols. First the notation and preliminaries, then each paper's assumptions, common ground and differences that will be used to analyze the efficiency and security of both protocols.

2.1 Notation and Bilinear-Group Preliminaries

Let p be a prime of bit length κ (the security parameter). Let G and G_T denote cyclic groups of order p equipped with an efficient, non-degenerate bilinear pairing $e : G \times G \to G_T$. Let $g \in G$ be a fixed generator. For composite order settings, we write $N = p_1 p_2$ with subgroups $G_1, G_2 \subset G$ of orders p_1, p_2 respectively, and exploit the fact that $e(g_1, g_2) = 1_{G_T}$ when $g_1 \in G_1$, $g_2 \in G_2$. All probability statements are over random points of the algorithms. Negligible functions are marked as $\mathrm{negl}(\kappa)$.

2.2 Assumptions

Abbrev.	**Name**	**Informal Statement**	**Used By**
DL	Discrete Logarithm	Given (g, g^a) output a.	Both papers (baseline hardness)
DDH	Decisional Diffie–Hellman	Distinguish (g^x, g^y, g^{xy}) from (g^x, g^y, g^z).	Benabbas et al. (base PRF)
t-SDH	t-strong Diffie-Hellman	From $(g, g^\alpha, \ldots, g^{\alpha^t})$ compute $(c, g^{1/(\alpha+c)})$.	KZG (binding)
d-SDDH	Decisional Skewed DDH	Decisional variant of SDH in one group.	Benabbas et al. (efficient PRF)
Subgroup Membership	–	Distinguish a random element of G from one of $G_2 \subset G$.	Benabbas et al. (verifiable DB)

Table 1: Summary of Assumptions

Both protocols depend on the hardness of discrete logarithm problems in pairing groups. KZG needs a structured reference string containing α powers; Benabbas et al. require only random generators, moving all structure into the secret PRF key.

With a strength ranking of DL $\subset$ DDH $\subset$ dSDDH/t-SDH, KZG works higher in the ladder to obtain constant communication. Benabbas et al. give the verifier the option of choosing between a weaker assumption with logarithmic client work or a stronger assumption with constant client work. For their verifiable database extension, Benabbas et al. move to composite-order groups and rely on the Bilinear Sub-Group Projection Assumption (BSGP), i.e., subgroup membership, while KZG operates in a prime-order setting.

Paper	**Component**	**Main Hardness Assumption**
KZG	Commit/Verify	DL, t-SDH, t-BSDH
Benabbas et al.	VC (basic)	DDH
	VC (fast)	d-SDDH
	Verifiable DB	Subgroup Membership

Table 2: Assumptions to Protocol Components Mapping

2.3 Algebraic PRFs and "Closed Form Efficiency" (CFE)

Benabbas et al. obtain sub-linear verification by masking coefficient vector $\mathbf{c}$ with a PRF $F_K(\cdot)$ whose outputs sit in the same group G and get a closed-form evaluation property: knowing the secret key K lets the client compute

$$R(x) = \sum_{i=0}^{d} F_K(i)x^i$$

in $O(\log d)$ or $O(1)$ time, depending on the PRF construction instead of $O(d)$.

Two instantiations are used:

- **Constant-Time PRF ($\mathbf{PRF_{fast}}$)** – secure under d-SDDH and enables $O(1)$ evaluation.
- **Logarithmic-Time PRF ($\mathbf{PRF_{light}}$)** – secure under DDH and has $O(\log d)$ work but with a weaker assumption.

These PRFs serve as a foundation for the paper's verifiable computation scheme.

2.4 Polynomial Commitment Assumptions (KZG)

KZG's constant size commitment $C = g^{\phi(\alpha)}$ relies on a trusted setup that publishes $(g, g^{\alpha}, \ldots, g^{\alpha^t})$.

Security reductions require:

- **t-SDH (binding)**: if an adversary can open the same C to two distinct polynomials, it solves t-SDH.
- **t-BSDH (batch opening)**: used when a single witness authenticates multiple evaluations simultaneously.
- **Hiding**: Computational under DL (or unconditional when fewer than t evaluations are revealed).

3 Constructions Overview

This section lays out how the two protocols work internally side by side. First, the verifiable delegation (VD) framework is covered, then KZG's polynomial commitment scheme, and finally the main security and efficiency comparisons.

3.1 VD over Large Datasets

Problem setting. A client holds a vector $\mathbf{a} \in \mathbb{F}_p^n$ but doesn't have the resources needed to evaluate a function F (a homogeneous polynomial or SQL-like query) on it. The client outsources the heavy work to an untrusted server and later wishes to verify (with sub-linear work) that the answer is correct.

Cryptographic primitive: algebraic PRF with CFE

Let

$$F_K(i) = g^{\alpha^i K} \in G$$

where $K \leftarrow \mathbb{F}_p$ is the secret key and (g, α) come from the public pairing setup. Because $F_K(i)$ is exponentially indexed, the client can compute

$$R(s) = \sum_{i=0}^{n-1} F_K(i)\, s^i$$

either in $O(\log n)$ using multi-exponentiation (under DDH) or in $O(1)$ using precomputed group elements g^{α^i} made public during setup (under d-SDDH).

Preprocessing (once per dataset)

1. Client picks PRF key K and a random scalar $r \in \mathbb{F}_p^\times$ that blinds the commitment to ensure unlinkability across multiple queries.
2. Forms masked polynomial $\widetilde{A}(X) = \sum_{i=0}^{n-1} a_i \cdot F_K(i) X^i$, and commits to it by sending the coefficient vector (or Merkle root) to the server with cost $O(n)$.
3. Publishes $\theta = r^{-1}$ so that later verifications can cancel the mask.

Online query (single evaluation)

For point $z \in \mathbb{F}_p$:

1. Server computes $y = \widetilde{A}(z)$ and the proof

$$\pi = g^{\frac{\widetilde{A}(\alpha) - y}{\alpha - z}} \in G$$

   The numerator is the value of the remainder polynomial $\frac{\widetilde{A}(X) - y}{X - z}$ evaluated at $X = \alpha$. It has a complexity of $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ communication.

2. Client verifies with one pairing check

$$e(\pi,\, g^{\alpha - z}) = e(g^y, g) \cdot e\left(\prod_i (g^{F_K(i)})^{a_i}, g\right)^\theta$$

   All group operations are constant, where the client's heavy work is the closed-form evaluation of $R(z)$, hidden in the final term.

There are other variants and extensions, such as re-randomizing r per query to protect earlier answers against adaptive queries. There are composite-order groups, where switching to $N = p_1 p_2$ allows verifiable keyword search and database updates via subgroup tests. There is also batch verification, where a single π can authenticate many points $\{z_j\}$ using the product of linear factors $\prod_j (\alpha - z_j)$, with a proof size of $\mathcal{O}(1)$ and server time $\mathcal{O}(n + \sum_j \deg_j)$.

3.2 Constant-Size Polynomial Commitments

Problem setting. A user commits to a degree $(d-1)$ polynomial $f(X)$ and later proves that $f(z) = y$ (or a batch of evaluations) to any verifier without revealing f.

Structured Reference String: A trusted setup outputs:

$$\text{CRS} = (g,\, g^{\alpha},\, g^{\alpha^2},\, \ldots,\, g^{\alpha^d}) \quad \text{for secret } \alpha \in \mathbb{F}_p$$

Commitment:

$$C = g^{f(\alpha)} \in G$$

Evaluation proof for a single point:

1. **Witness:**
$$W = g^{\frac{f(\alpha)-y}{\alpha - z}} \quad (\text{since } f(X) - y = (X - z) \cdot q(X))$$

2. **Verification:**
$$e(C/g^{y},\, g) = e(W,\, g^{\alpha - z}) \quad \Longleftrightarrow \quad f(z) = y$$
Both sides consist of a single pairing product in constant time and communication.

Batch opening to t points: Let points be $z_1, \ldots, z_t$ with values $y_j = f(z_j)$.

- **Witness polynomial:**
$$q(X) = \frac{f(X) - \sum_j y_j\, L_j(X)}{\prod_j (X - z_j)}$$
where L_j are Lagrange basis polynomials.

- **Witness element:** $W = g^{q(\alpha)}$

- **Verification:**
$$e\left(C/g^{\sum_j y_j L_j(\alpha)},\, g\right) = e\left(W,\, g^{\prod_j (\alpha - z_j)}\right)$$

Proof size and verifier work remain $O(1)$ where the prover's time is $O(d + t \log^2 t)$ using fast multipoint evaluation.

3.3 Side-by-Side Comparison

Criterion	**Verifiable Delegation (VD)**	**KZG Polynomial Commitments (PC)**
Verifier time	$O(1)$ group operations and closed-form PRF eval ($O(\log n)$ or $O(1)$ under d-SDDH)	$O(1)$
Prover time	$O(n)$ per query	$O(d)$ single point, $O(d + t\log^2 t)$ batch
Proof size / Commitment	1 group element (π) and 1 field element y	1 group element (W) and 1 field element y
Trusted setup	None	Yes (powers of secret α)
Assumptions	DDH or d-SDDH	t-SDH (plus DL for hiding)
Dynamic data	Natural (server stores vector)	Needs update protocol
Universal verification	Private-key (knowledge of K is needed)	Public-key

Table 3: Comparison of Verifiable Delegation vs. Polynomial Commitments

Interpretation. In terms of efficiency, KZG is lighter for the prover when handling low-degree polynomials or batch queries. VD is better when the dataset is huge and the preprocessing amortizes over many evaluations. In security, VD lets the client choose the assumption strength versus local effort and avoids any trusted setup with verification done in private. KZG attains public verifiability and unconditional binding (regarding the CRS) but takes over the cost of a trusted setup. When frequent updates or keyword searches on a large database are required, VD's composite-order extension is more fitting. KZG is better when compact proofs and blockchain transparency matter.

4 Applications

This section analyses the concrete problems each paper targets, shows how the underlying math is instantiated, and then maps the schemes to more general deployment scenarios.

4.1 Applications of KZG

Domain	How KZG is used	Why KZG is a good fit
Verifiable Secret Sharing (VSS)	The dealer commits once to the degree-t sharing polynomial f by publishing $C = g^{f(\alpha)}$. Each share $s_i = f(i)$ is opened with a witness $W_i = g^{\frac{f(\alpha)-s_i}{\alpha-i}}$ and every participant checks a single pairing $e(C/g^{s_i}, g) = e(W_i, g^{\alpha-i})$.	Constant-size broadcast instead of $O(n)$ coefficient commitments.
Zero-Knowledge Sets / Databases	Commit to the vanishing polynomial $\phi(X) = \prod_{x \in S}(X - x)$. Membership proofs use usual KZG point openings and non-membership uses the cofactor polynomial $\phi/(X - x)$.	Proofs are constant size and reveal nothing beyond (non-)membership beating Merkle trees $O(\log n)$ paths.
Selective-Disclosure Credentials & Content-Extraction Signatures	A credential signs a list of attributes packed into a polynomial. The holder later shows only the subset needed for a transaction by opening the commitment at the relevant points with aggregated proofs.	Communication grows with number of revealed fields k rather than the total t.

Table 4: Applications of KZG Commitments

The important mathematical concept is every application reduces to proving

$$e(C/g^y, g) = e(W, g^{\alpha-z})$$

so KZG inherits the same flexible interface regardless of whether y is a secret share, a set membership flag or a field element.

4.2 Applications of Verifiable Delegation (VD)

Domain	How VD is used	Why VD is a good fit
Proofs of Retrievability	The file is viewed as a coefficient vector $\mathbf{a}$. For each challenge, the server returns $y = \widetilde{A}(r)$ and proof $\pi = g^{(\widetilde{A}(\alpha)-y)/(\alpha-r)}$. The client verifies with one pairing and a PRF evaluation.	Audit streams of gigabyte-sized data with constant client work
Verifiable Keyword Search	Keywords are embedded as polynomial roots where a server showing a match produces the same (y, π) proof at the query point chosen by the client.	Handles private searches while the data owner keeps the PRF key confidential.
Verifiable Databases	In the composite-order variant, each table entry is allocated to a subgroup where updates change only the corresponding exponent so the commitment doesn't need be recomputed. Proofs remain one group element.	Supports unbounded updates with $O(1)$ verifier overhead (KZG can do only with extra accumulator logic).

Table 5: Applications of Verifiable Delegation

The important mathematical concept is VD turns every outsourced answer into

$$e(\pi, g^{\alpha - z}) \stackrel{?}{=} e(g^y \cdot R(z), g)$$

where $R(z)$ is the closed-form sum of algebraic PRF outputs. Because R is computable in $O(\log n)$ (DDH) or $O(1)$ (d-SDDH), the verifier's work stays sub linear no matter how large the dataset.

4.3 Choosing the Right Tool

Application	Better with KZG	Better with VD
Public verifiability / many third-party auditors	Witness & commitment are self-contained and anyone can check	Requires secret PRF key
Dynamic data (frequent updates)	Needs extra vector commitment machinery	Composite-order verifiable databases handles updates
Gigantic datasets, repeated queries	Prover's $O(d)$ time per query becomes expensive	Server cost $O(n)$ once and verifier $O(1)$
No trusted setup allowed	Depends on powers of α CRS	Plain pairing parameters are enough
Blockchain / roll-up proofs	Widely deployed	Specialized solution because private-key verification is needed

Table 6: KZG vs VD: Best Use Cases

In general, **KZG** should be used when you have to publish proofs to the public where constant proof size and public checking outweighs a one time trusted setup. **VD** should be used when a single data owner repeatedly queries a changing, massive dataset because the server's linear time preprocessing amortizes and the owner doesn't have to do any ceremony. These real world schemes show how the same algebraic identity changes to serve very different operational circumstances.

5 Discussion

The two protocols have different security strength, scalability, dependence on a trusted setup and adaptability to dynamic data. This section will discuss the differences of the two schemes.

Security Model Strength

KZG relies on the t-Strong Diffie-Hellman (t-SDH) family of assumptions in pairing-friendly groups. These assumptions are stronger than basic discrete log and still within the generic group model. On the other hand, VD allows the client to choose between different assumption levels with an optional composite-order group extension adding only a subgroup membership assumption. When a key is exposed, KZG is easier to break. If the setup secret α is leaked at any point then all the past proofs are compromised. Whereas VD limits damage to the verifier's own context and isolates the risk of key exposure.

In verifiability, KZG supports public verification by allowing any third party to validate proofs. VD narrows verifiability to the client with the private PRF key making VD more useful in private outsourcing. Something KZG lacks is VD's use of randomized masks per query which gives a stronger resistance to adaptive and selective failure attacks.

Scalability

In both schemas the verifier work has two pairings but VD may add PRF computations depending on the configuration of DL or constant time. KZG proofs are faster to produce due to polynomial evaluation optimizations requiring only $\mathcal{O}(d \log d)$. VD uses streaming dot products in $\mathcal{O}(n)$ and stores both coefficients and PRF outputs. While KZG doesn'y offer amortization (each query recomputes the witness), VD allows multiple queries to reuse preprocessing.

These characteristics explain real-world choices. KZG is dominant in blockchain rollups where compact proofs for kilobyte-scale data are crucial and VD in archival systems like Proofs of Retrievability at the petabyte scale where amortization is key.

Trusted Setup

KZG dictates a structured reference string derived from a trapdoor α, usually generated by a ceremony. VD avoids this by requiring only a standard pairing group, avoiding ceremonies and their environmental or security implications. This comes at the cost of giving up public verifiability and accepting stronger cryptographic assumptions. For organizations subject to regulation, the lack of a trusted setup makes VD more favorable even with its performance trade-offs.

Adaptability to Dynamic Data

KZG binds the entire polynomial to a single commitment point $f(\alpha)$ which makes updates costly. In the paper, VD requires sets to be fized during preprocessing but shows support for localized updates by altering only the subgroup exponent of the affected record.

Summary

If the trusted setup can be accepted, KZG is ideal when small proof size, public auditability, and blockchain compatibility are important. VD is better for massive datasets, where setup-free deployment, amortized queries, and private verification are favorable. In practice, these schemes are not mutually exclusive and hybrid deployments could be used.

6 Conclusion

This comparative study set out to determine how two protocols solve the common problem of verifying outsourced computation. Both schemes reduce verification to a single bilinear-pairing identity and give constant-size proofs. They also greatly diverge in their system assumptions, performance, and deployment logic.

The protocols meet where each construction encodes correctness in the same quotient witness $W = g^{\frac{f(\alpha)-y}{\alpha-z}}$ and validates it with a two-pairing check $e(C/g^y, g) = e(W, g^{\alpha-z})$. This shows the power of pairings for succinct proof systems. They both give the client or $\mathcal{O}(1)$ auditor a fixed cost test independent of data magnitude and allows edge devices to act as verifiers. Furthermore, their quotient proofs aggregate naturally since KZG has Lagrange interpolation and VD has multiplying linear factors. This shared pattern builds multi-point openings, streamed audits, and keyword searches.

The main distinctions happen when KZG orders a one time ceremony to publish $(g, g^\alpha, \ldots)$ and VD works with an unstructured pairing group and a private PRF key. The result is public verifiability for KZG versus a local verifier correctness for VD.

There are different hardness assumptions. KZG relies on the stronger t-SDH (and t-BSDH) assumptions to achieve constant-time witnesses and VD lets the client trade as-

sumptions (DDH vs. d-SDDH) against logarithmic or constant client work. The prover has also different amount of work where VD's server cost is linear in the dataset but paid once and then amortized over many queries. KZG requires a new $\mathcal{O}(d)$ (or $d \log d$) polynomial evaluation per opening. This makes KZG ideal for modest degrees but worse for petabyte archives.

There are different scenarios to consider when choosing when to deploy either scheme. KZG should be used when you need *public transparency and tiny proofs*. VD should be used when you outsource computation to a single auditor who controls the verification key and must audit a *massive amount of data*. A hybrid option could be used when a storage provider might use VD for daily integrity audits while posting a KZG commitment of each day's hash tree to a public ledger essentially blending setup-free scalability with public accountability.

To summarize, KZG and VD represent two complementary points on verifiable outsourcing. Their common algebraic skeleton asserts how pairings are very useful. Still, the different choices regarding setup, assumptions, and adaptability tailor them to very different operational needs. The people who use these schemes should consider the presence of third party auditors, tolerance for trusted setups, dataset mutability, and amortisation potential when choosing or combining these techniques. Advancements in polynomial commitment schemes and PRF constructions will likely dwindle some security constraints and increase their applicability as time keeps going.

7 References

References

@miscbenabbas2011verifiable, author = Siavosh Benabbas and Rosario Gennaro and Yevgeniy Vahlis, title = Verifiable Delegation of Computation over Large Datasets, howpublished = *Cryptology ePrint Archive*, Report 2011/132, year = 2011, url = https://eprint.iacr.org/2011/132, note = Available at: `https://eprint.iacr.org/2011/132`

@inproceedingskzg2010polynomial, author = Aniket Kate and Gregory M. Zaverucha and Ian Goldberg, title = Constant-Size Commitments to Polynomials and Their Applications, booktitle = Advances in Cryptology – ASIACRYPT 2010, series = Lecture Notes in Computer Science, year = 2010, publisher = Springer, url = https://eprint.iacr.org/2010/486